Assignment 1 [CO1]

Metro Smart-Card Fare System

Course: DSA0101 - OOP with C++ **Student:** Sarathy.B **Register number:** 1925260047 **Repository:** DSA0101-192524005 **Language:** C++11 **Compiler used for recorded runs:** LLVM-MinGW / Clang 23.1.1

Abstract

This assignment models a metro smart-card fare system. It covers object-oriented design, C++ tokens and types, conversions, operators, fare rules, input validation, testing, numeric precision, and a design change for Airport fares and monthly passes. The programs use integer paise for money and check that a paid trip leaves at least Rs 50 on the card.

1. Problem model and assumptions

- Zones are Inner, Middle, and Outer. Task 12 adds Airport.
- Base fares are Rs 10, Rs 20, and Rs 30; Airport costs Rs 60.
- Peak hours are 08:00 to before 10:00 and 17:00 to before 20:00. The peak surcharge is 20%.
- Money is stored as integer paise. A paid trip must leave at least 5000 paise (Rs 50).
- The assignment does not give a student or senior concession rate, so the executable tasks do not invent one. Any50% rate mentioned in design notes is only an example.
- Task 12 applies the 10% weekend discount instead of the peak surcharge. A covered monthly-pass trip costs zero.

The work is in the subject repository selected for this assignment. The handout's separate repository naming pattern may need confirmation with the instructor.

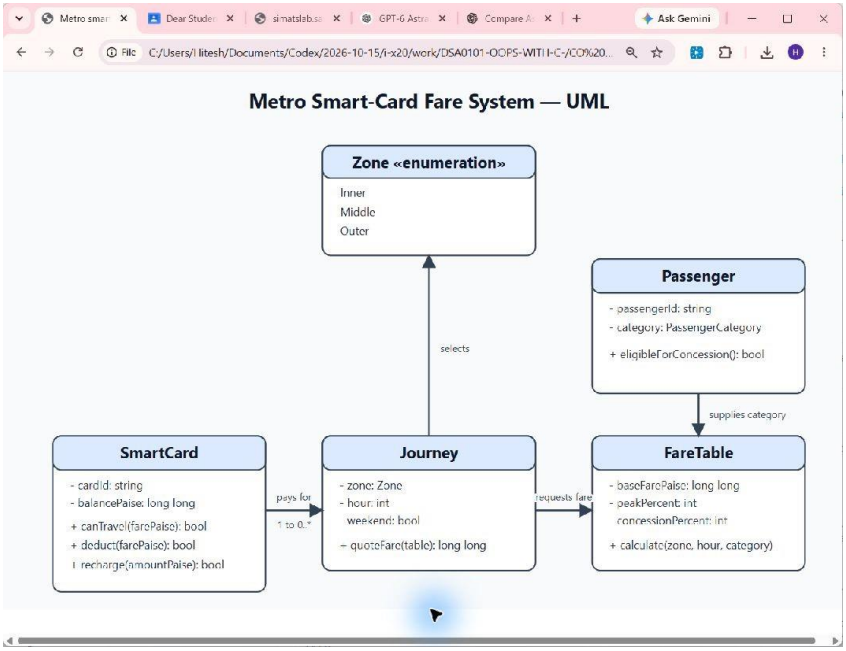
2. Task 1 - Objects and OOP design

Object	State and responsibility
`SmartCard`	Card ID and private balance; checks travel, deducts fare, and recharges safely.
`Passenger`	Passenger ID and category; identifies the applicable concession policy.
`Journey`	Zone and departure time; requests a fare and payment.
`FareTable`	Base fares and fare rules; calculates the fare in one place.
`Zone`	A fixed set of named zones represented with `enum class`.

Encapsulation keeps the balance private. Abstraction exposes operations such as `calculateFare` and `deduct` without exposing every calculation. Inheritance or polymorphism could support more fare policies later, but a category and a central fare policy are enough for this small example.

The main rules are: reject an invalid zone or time, never allow a negative balance, leave at least Rs 50 after a paid trip, and do not change the balance when a trip is declined. Fare lookup and card approval are O(1) per trip; processing `n` trips is O(n). A procedural version can use structs and free functions, but every caller must follow the balance checks.

Rendered UML class diagram



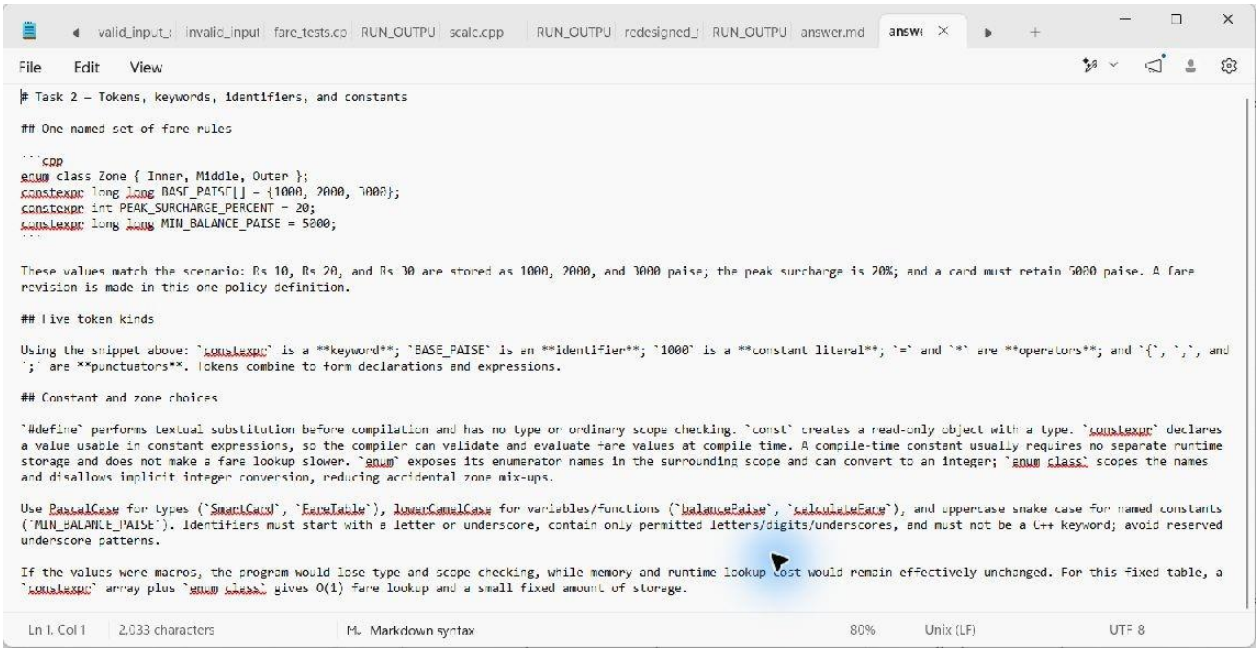
Detailed answer: Task01/answer.md

3. Task 2 - Tokens and constants

```
enum class Zone { Inner, Middle, Outer }; constexpr
long long BASE_PAISE[] = {1000, 2000, 3000}; constexpr
int PEAK_SURCHARGE_PERCENT = 20; constexpr long long
MIN_BALANCE_PAISE = 5000;
```

In this example, `constexpr` is a keyword, `BASE_PAISE` is an identifier, `1000` is a literal, `=` and `*` are operators, and braces and semicolons are punctuators. `constexpr` makes fixed fare values available at compile time. `enum class` keeps zone names scoped and prevents accidental integer conversion. These choices are safer than text macros such as `#define`.

Task 2 answer evidence



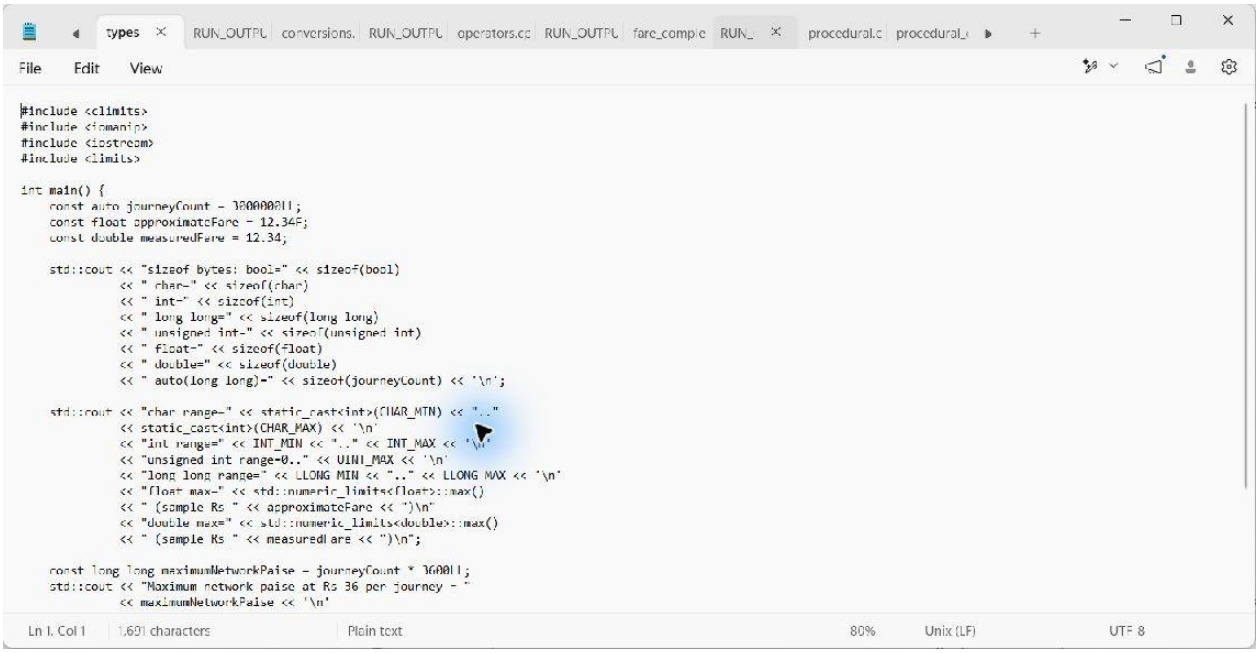
Detailed answer: Task02/answer.md

4. Task 3 - Data types and compatibility

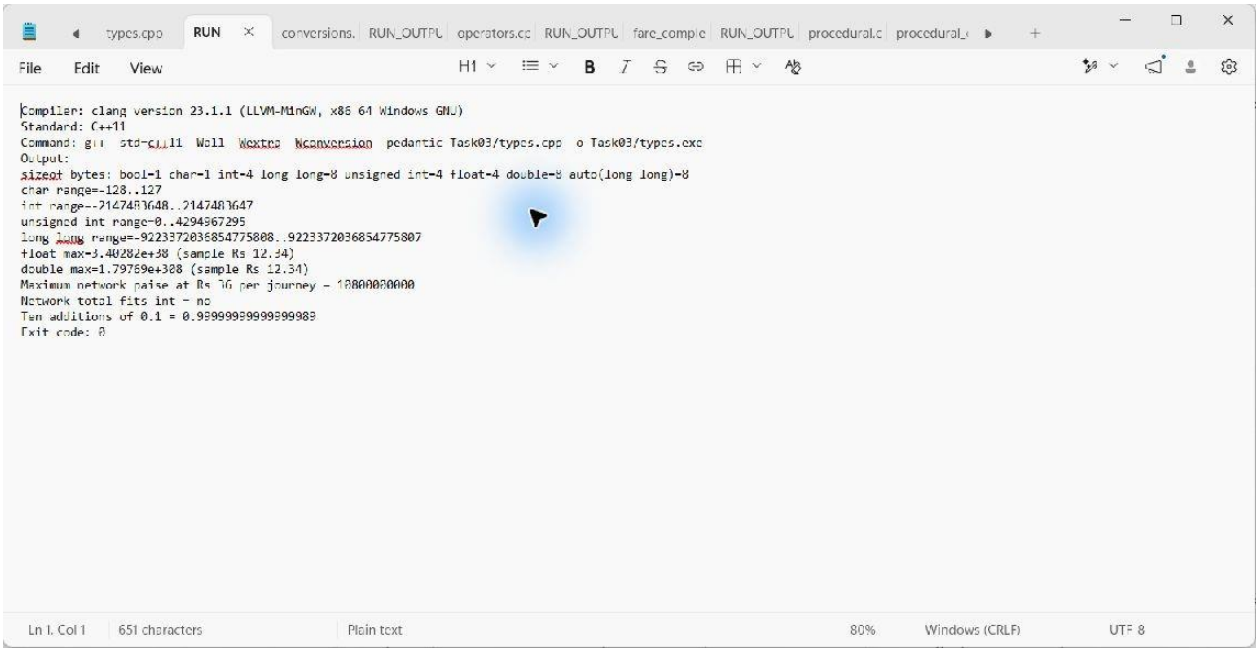
Value	Type	Reason
Fare and balance	`long long` in paise	Exact money arithmetic and sufficient range for network totals.
Hour and percentage	`int`	Small whole values; signed input makes negative values easy to reject.
Zone	`enum class Zone`	Named values prevent mixing zones with unrelated integers.
Approval flag	`bool`	Represents true or false.
Non-negative count	`unsigned int`	Useful only after validating signed input; unsigned subtraction can wrap.
Inferred local value	`auto`	Deduces the initializer's type at compile time.
Approximate measurement	`float` or `double`	Useful for demonstration, not for storing fares.

On the tested compiler, the measured sizes were: `bool` 1 byte, `char` 1, `int` 4, `unsigned int` 4, `long long` 8, `float` 4, and `double` 8. These sizes depend on the compiler. The `int` range was -2,147,483,648 to 2,147,483,647; the `unsigned int` range was 0 to 4,294,967,295. Three million trips at the highest peak fare total 10,800,000,000 paise, which does not fit in a 32-bit `int`.

Task 3 source evidence



Task 3 output evidence



Source: Task03/types.cpp. Detailed answer: Task03/answer.md.

5. Task 4 - Implicit conversions

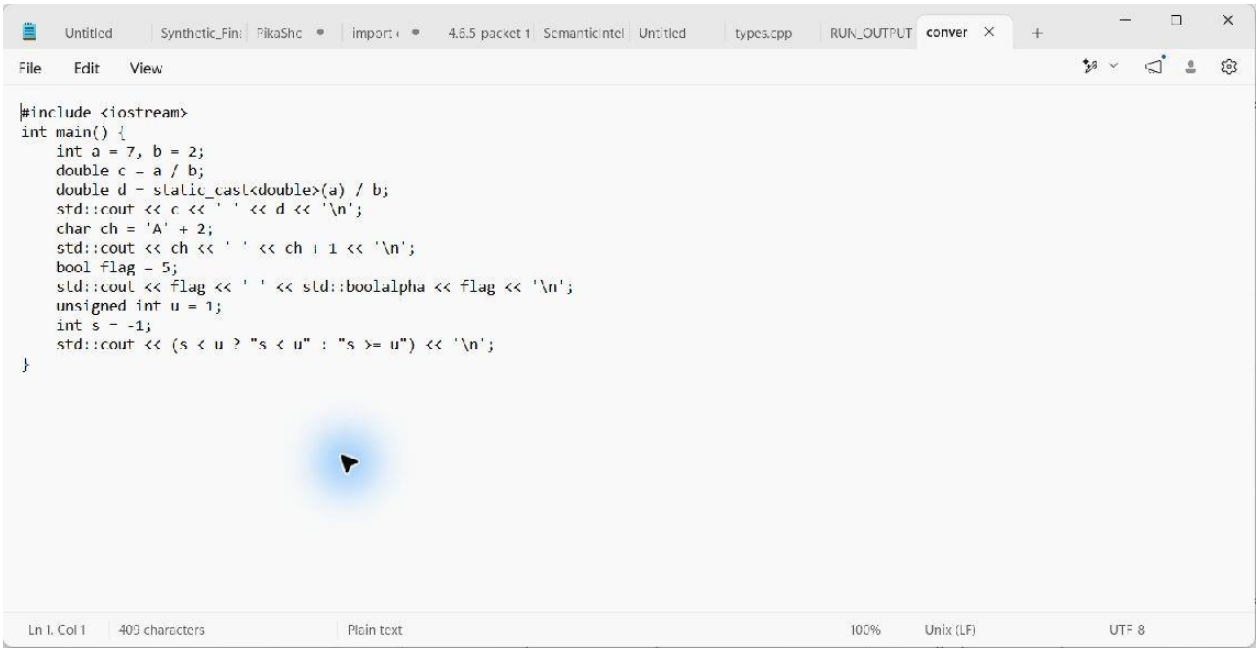
The program shows integer division, an explicit cast, character promotion, conversion to `bool`, and a signed/unsigned comparison. Integer division makes `7 / 2` equal 3; casting before division gives 3.5. A `char` is promoted to `int` in arithmetic. A nonzero integer converts to `true`. In the signed/unsigned comparison, -1 converts to an unsigned value, so the comparison is false on the tested system. The compiler reports the expected signed/unsigned warning.

Reference prediction and actual output:

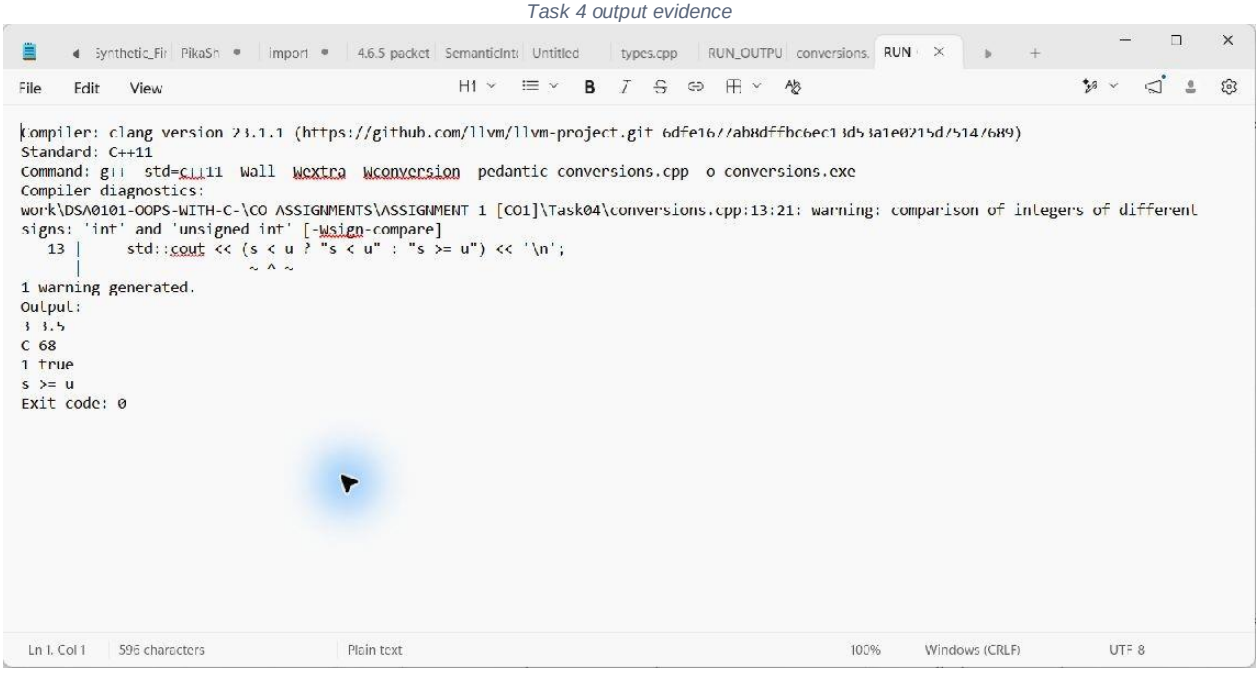
```
3 3.5
C 68 1
true s
>= u
```

The output matches the stored reference prediction. The prediction was prepared with assistance, so it should not be described as an independent pre-run prediction.

Task 4 source evidence



Task 4 output evidence



Source: Task04/conversions.cpp. Detailed answer: Task04/answer.md.

6. Task 5 - Operators and precedence

The program demonstrates precedence, parentheses, pre-increment and post-increment, the conditional operator, integer division, remainder, and a loop. The peak check is:

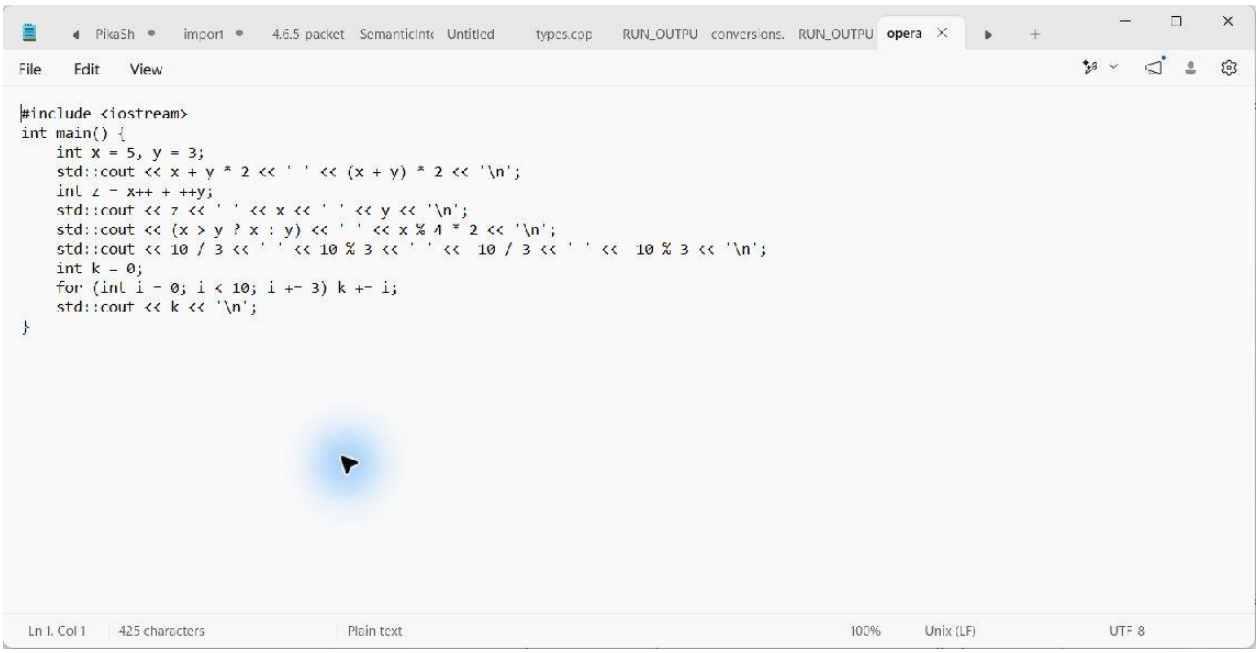
```
(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)
```

Actual output:

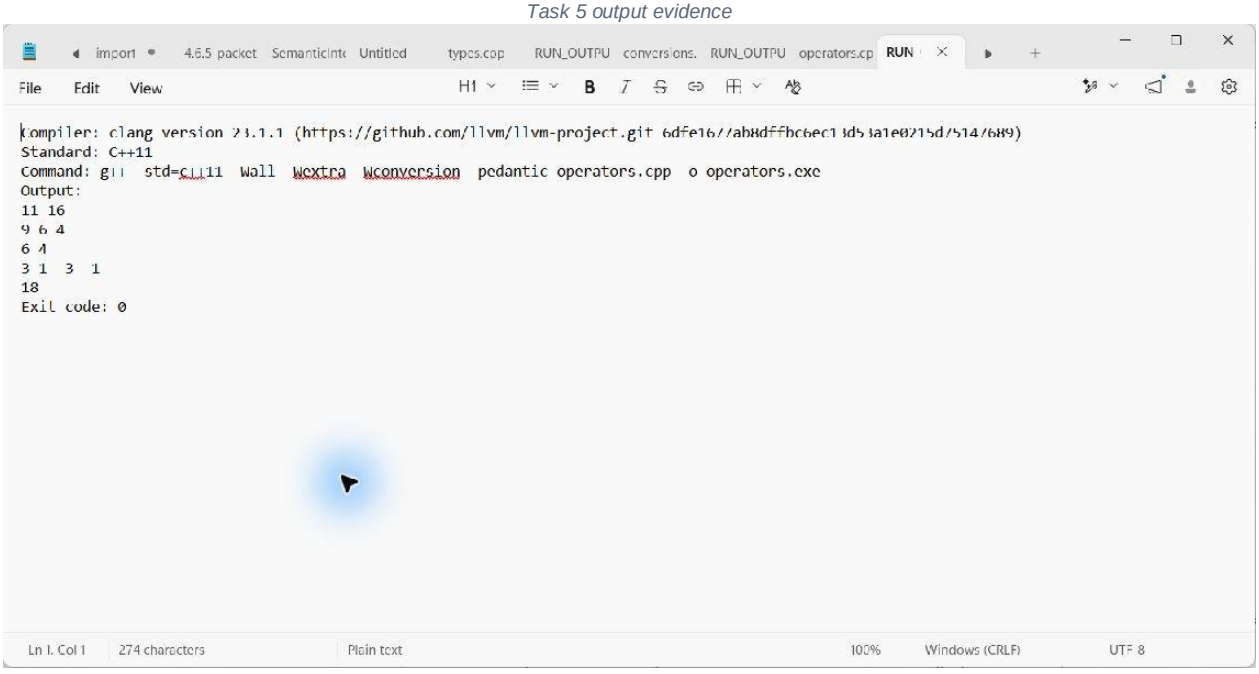
```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

The output matches the stored reference prediction. The prediction was prepared with assistance. The range expression takes O(1) time and O(1) space.

Task 5 source evidence



Task 5 output evidence



Source: Task05/operators.cpp. Detailed answer: Task05/answer.md.

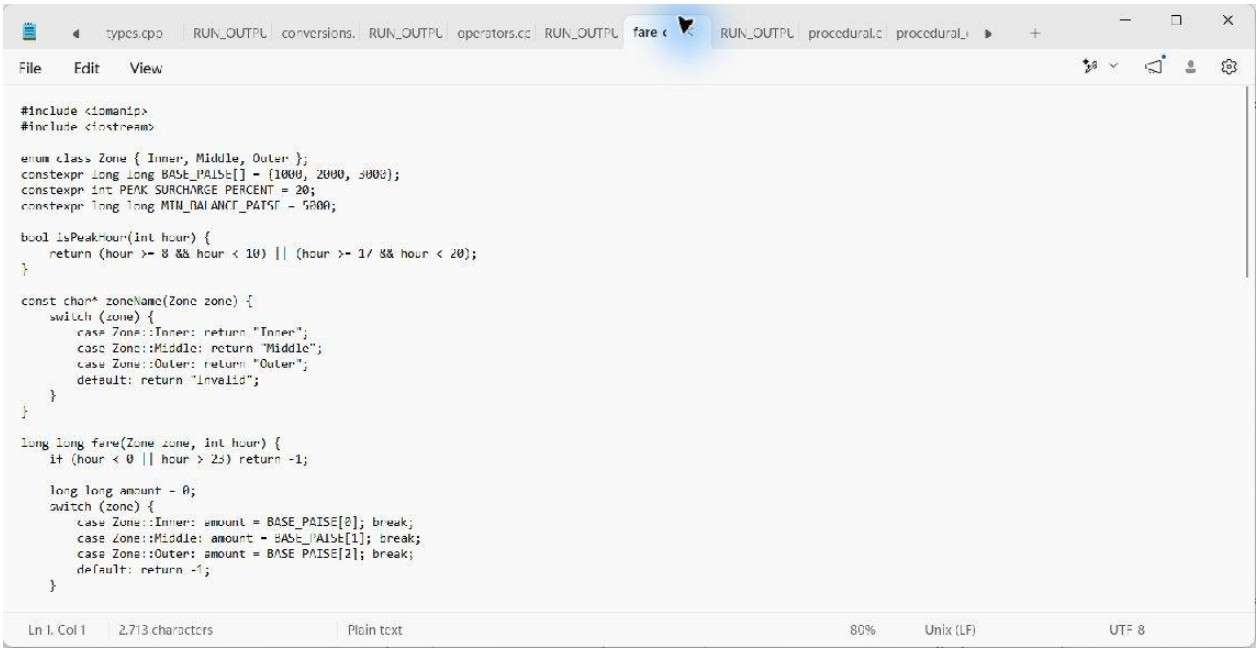
7. Task 6 - Control structures

The program validates the zone and hour, selects the base fare with `switch`, applies the peak surcharge with `if`, and deducts the fare only if the remaining balance is at least Rs 50. It also checks the end times of both peak periods.

```
For each journey:
    Reject an invalid zone or hour.
    Select the base fare for the zone.
    Mark the hour as peak only in 08:00-09:59 or 17:00-19:59.
    Add 20 percent during peak hours.
    If balance - fare is at least 5000 paise, deduct and approve.
    Otherwise, decline and leave the balance unchanged.
```

The sample Inner off-peak trip costs Rs 10 and leaves Rs 90. An Outer peak trip costs Rs 36 and leaves Rs 54. The next trip is declined because it would leave less than Rs 50. Exactly 10:00 and 20:00 are off-peak: their Inner and Outer fares are Rs 10 and Rs 30.

Task 6 source evidence



```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

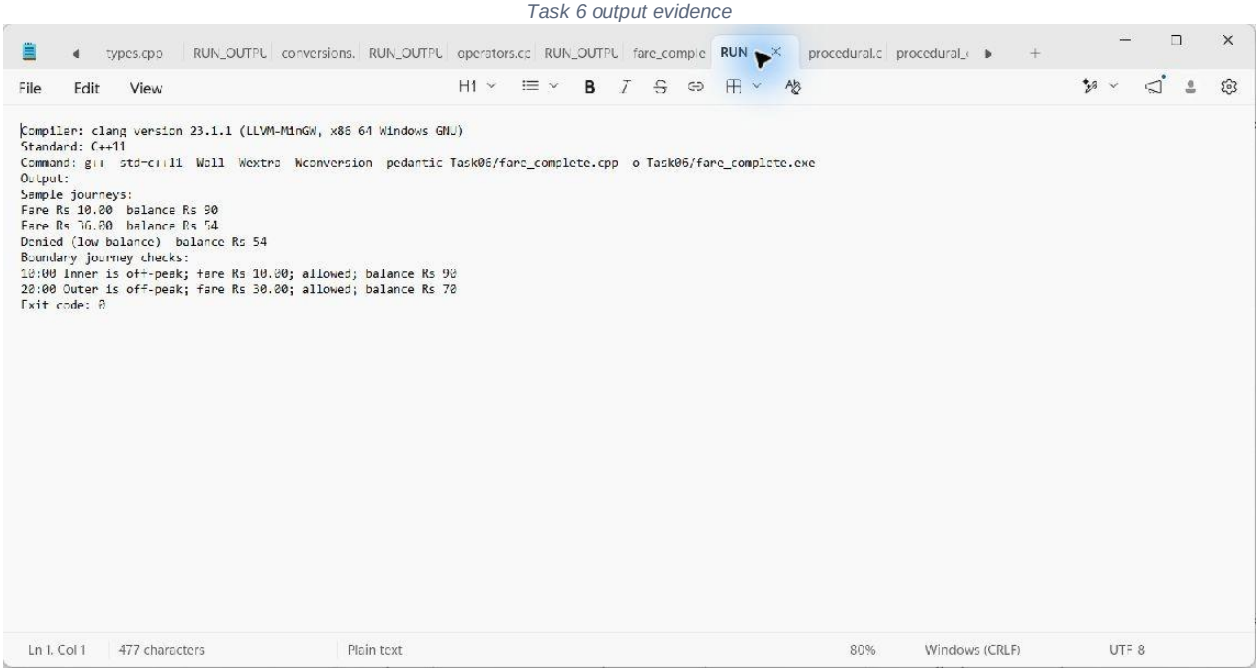
bool isPeakHour(int hour) {
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

const char* zoneName(Zone zone) {
    switch (zone) {
        case Zone::Inner: return "Inner";
        case Zone::Middle: return "Middle";
        case Zone::Outer: return "Outer";
        default: return "Invalid";
    }
}

long long fare(Zone zone, int hour) {
    if (hour < 0 || hour > 23) return -1;

    long long amount = 0;
    switch (zone) {
        case Zone::Inner: amount = BASE_PAISE[0]; break;
        case Zone::Middle: amount = BASE_PAISE[1]; break;
        case Zone::Outer: amount = BASE_PAISE[2]; break;
        default: return -1;
    }
```

Task 6 output evidence



```
Compiler: clang version 23.1.1 (LLVM-MinGW, x86_64 Windows GNU)
Standard: C++11
Command: g++ -std=c++11 -Wall -Wextra -Wconversion -pedantic Task06/fare_complete.cpp -o Task06/fare_complete.exe
Output:
Sample journeys:
Fare Rs 10.00  balance Rs 90
Fare Rs 36.00  balance Rs 54
Denied (low balance)  balance Rs 54
Boundary journey checks:
10:00 Inner is off-peak; fare Rs 10.00; allowed; balance Rs 90
20:00 Outer is off-peak; fare Rs 30.00; allowed; balance Rs 70
Exit code: 0
```

Source and pseudocode: Task06/fare_complete.cpp, Task06/pseudocode.txt.

8. Task 7 - Procedural and object-oriented versions

Both versions begin with 10,000 paise and try to pay 3,600 paise. Both approve and leave 6,400 paise.

Procedural version	Object-oriented version
Balance is a variable passed to a function.	`SmartCard` stores the balance privately.

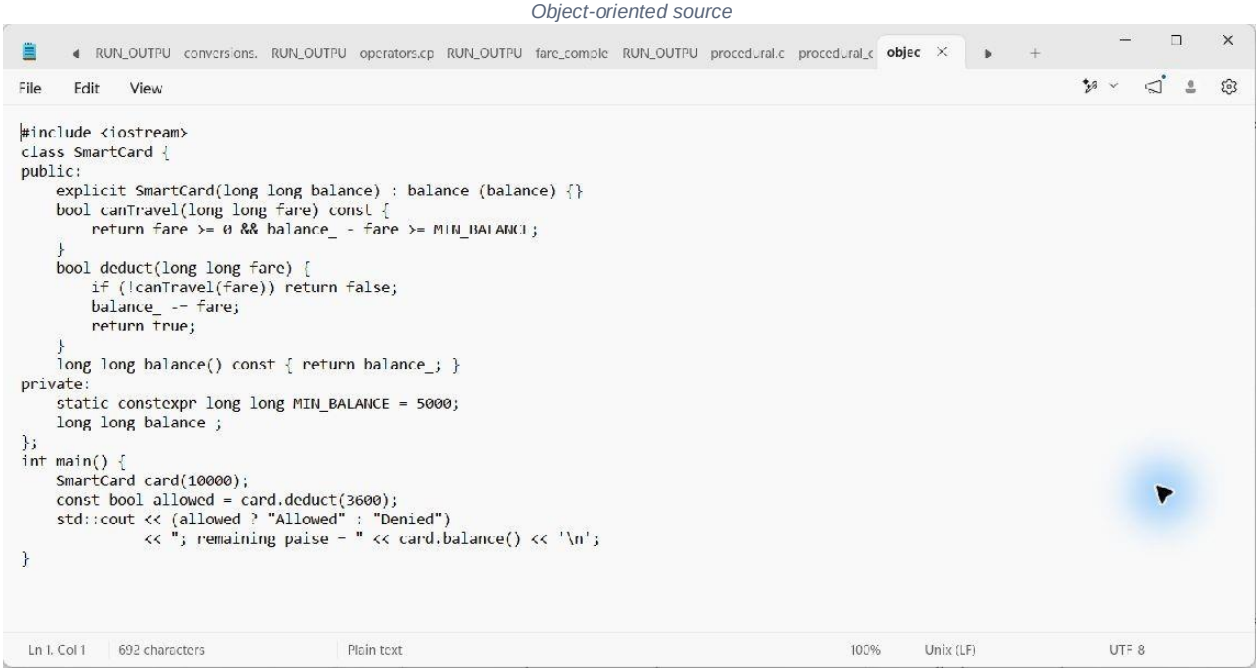
Procedural version	Object-oriented version
The caller must use the checking function.	The card method keeps the balance rule with the card.
Small and direct for a simple task.	Easier to protect the rule as the system grows.

Each check and deduction is O(1), and each card stores O(1) state.

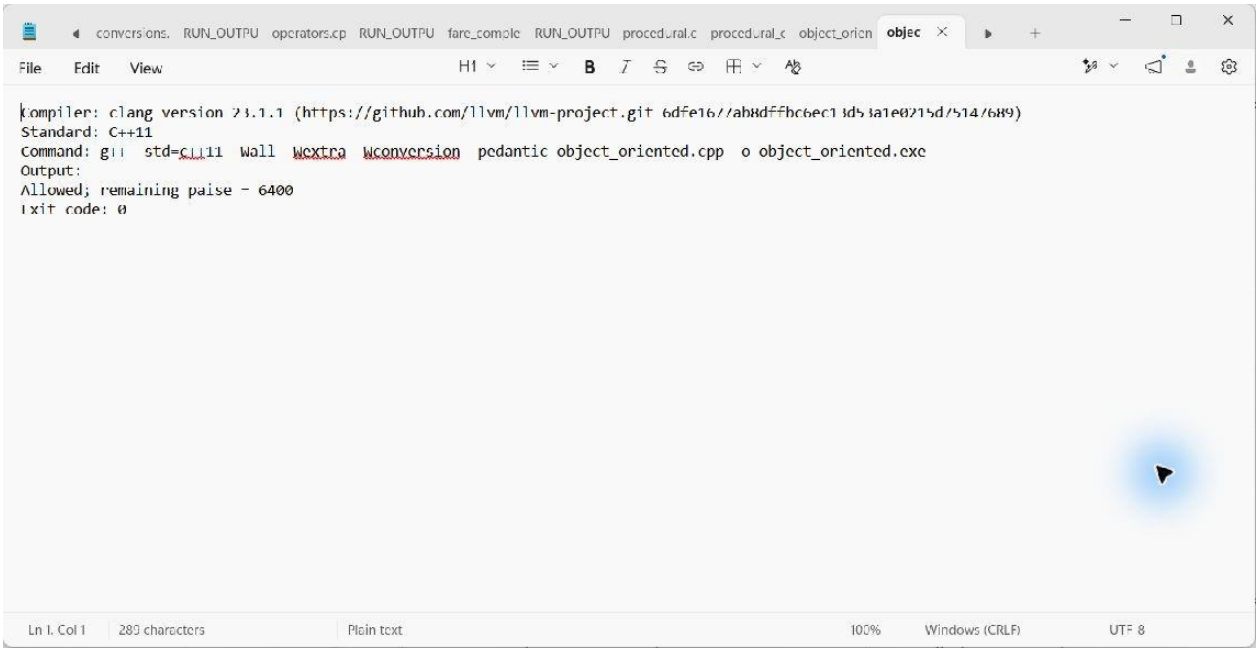
Procedural source



Object-oriented source



Task 7 output



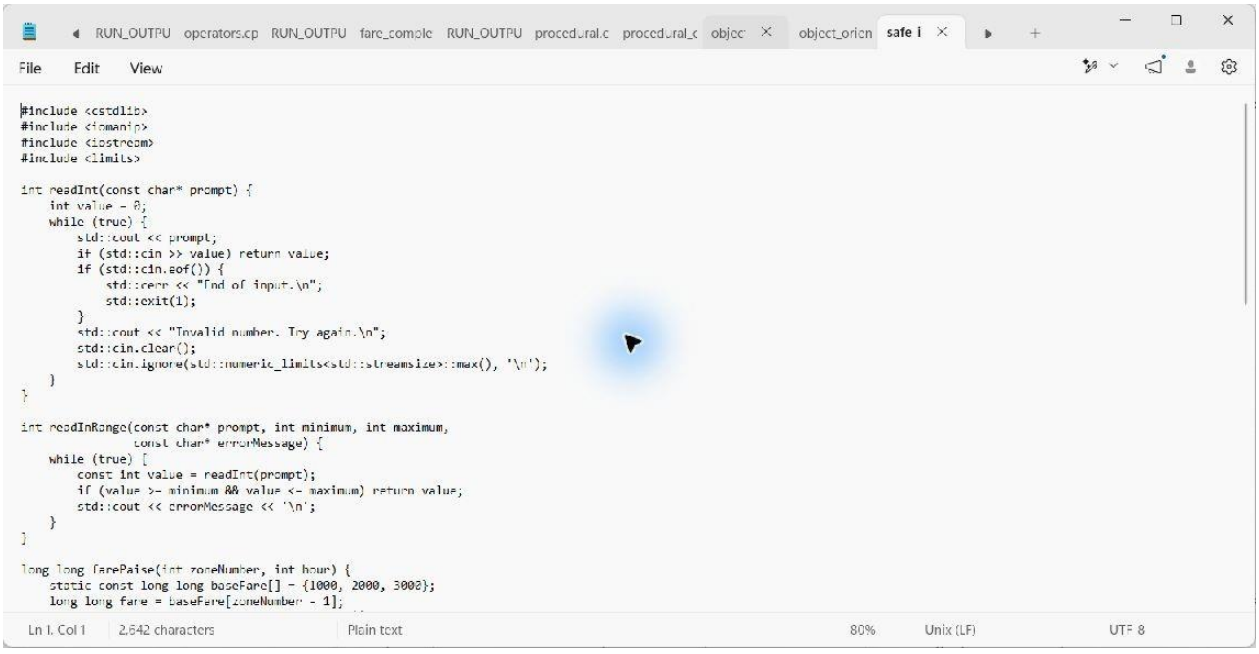
Sources: procedural.cpp and object_oriented.cpp.

9. Task 8 - Safe input and output

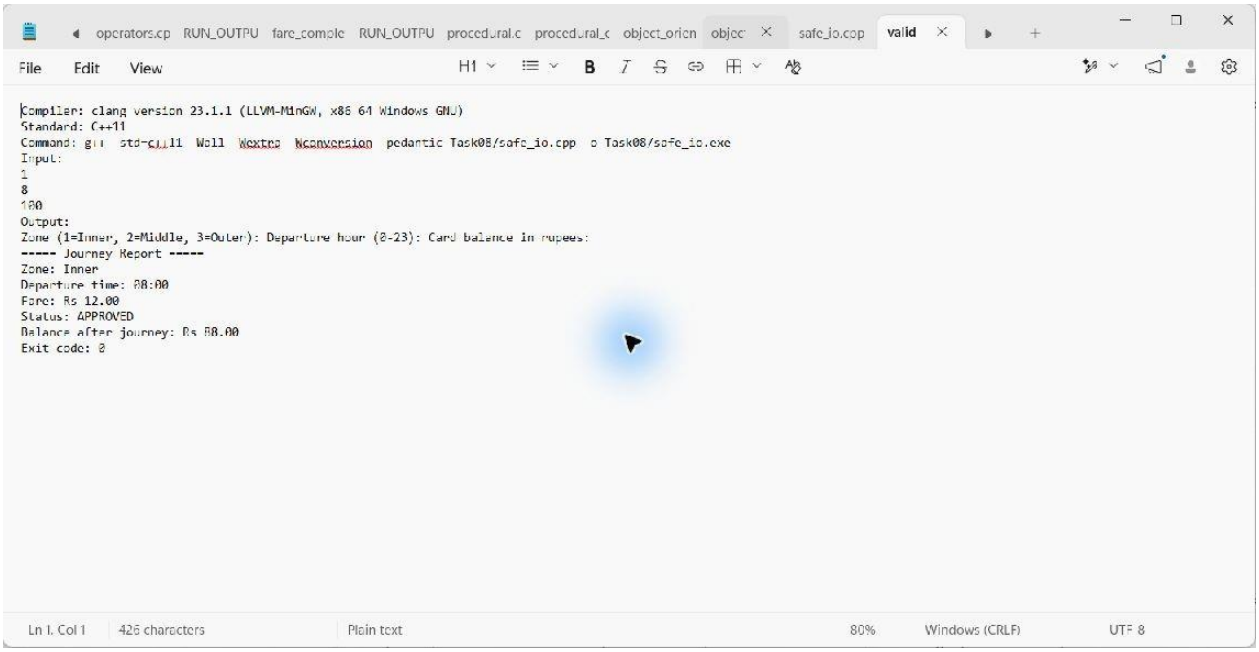
The program retries after nonnumeric input, rejects zones outside 1-3 and hours outside 0-23, and rejects a negative balance. It prints the zone, time, fare, approval status, and remaining balance using two decimal places.

For valid input `1, 8, 100`, the Inner-zone peak fare is Rs 12.00. The trip is approved and the remaining balance is Rs 88.00. The recovery run rejects a nonnumeric zone, zone 4, hour 24, and a negative balance before accepting valid input.

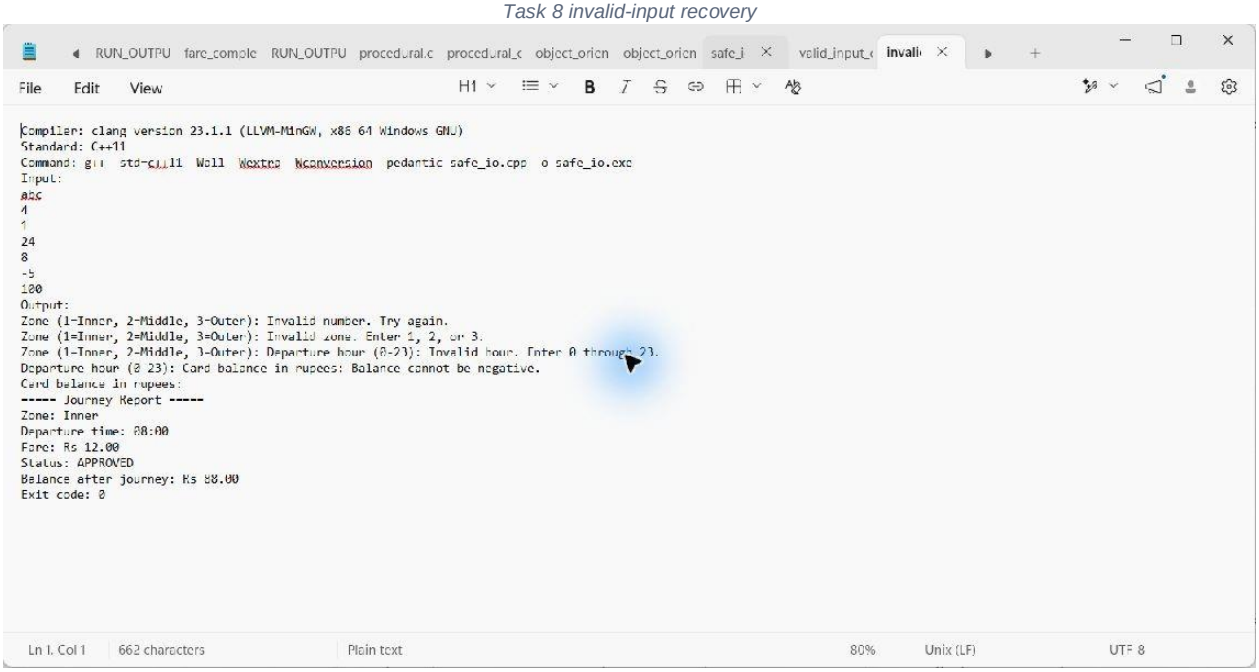
Task 8 source evidence



Task 8 valid-input output



Task 8 invalid-input recovery



Source and inputs: safe_io.cpp, valid_input.txt, invalid_input.txt.

10. Task 9 - Fare logic tests

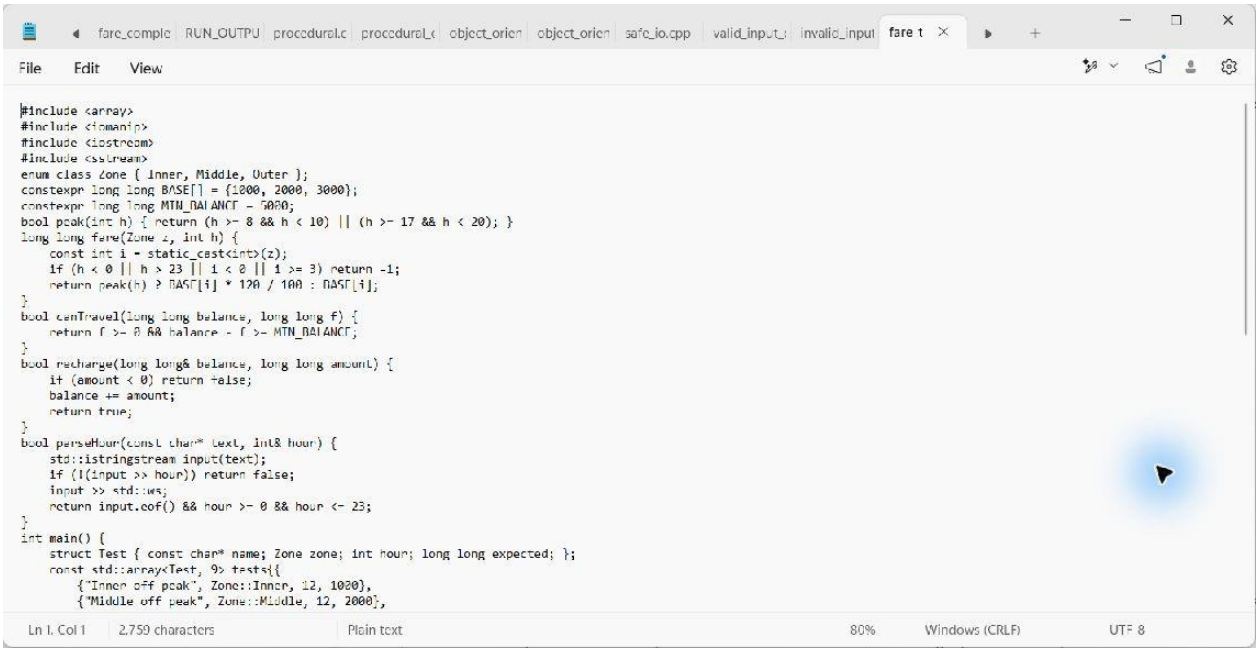
The test program checks fares, peak boundaries, invalid values, the minimum balance, negative recharge, and the total of five trips.

Test	Expected	Result
Inner, Middle, Outer off-peak	1000, 2000, 3000 paise	Pass
08:00 / 10:00	1200 / 1000 paise	Pass
17:00 / 20:00	2400-3600 / 3000 paise	Pass

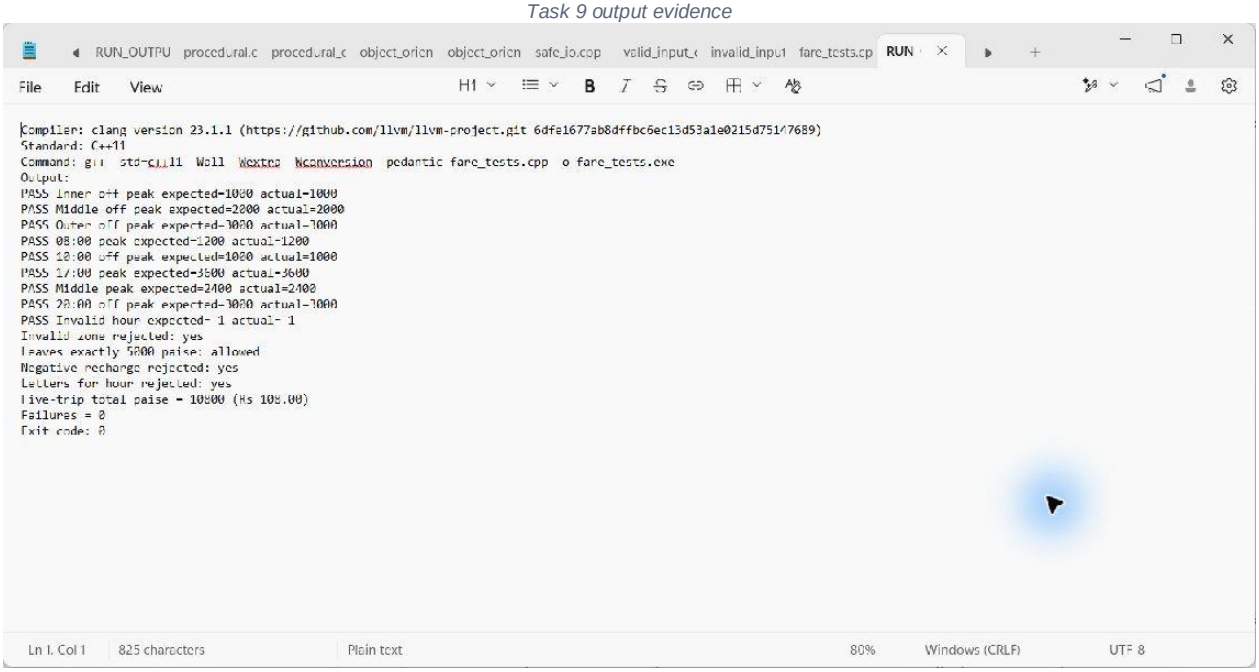
Test	Expected	Result
Invalid zone or hour	Rejected	Pass
Balance 6200 paying 1200	Allowed, leaves 5000	Pass
Negative recharge and text hour	Rejected	Pass
Five-trip total	10800 paise	Pass

The recorded run reports `Failures = 0`. For `t` test cases, the test loop takes O(t) time.

Task 9 source evidence



Task 9 output evidence



Source and case list: fare_tests.cpp, test_plan.csv.

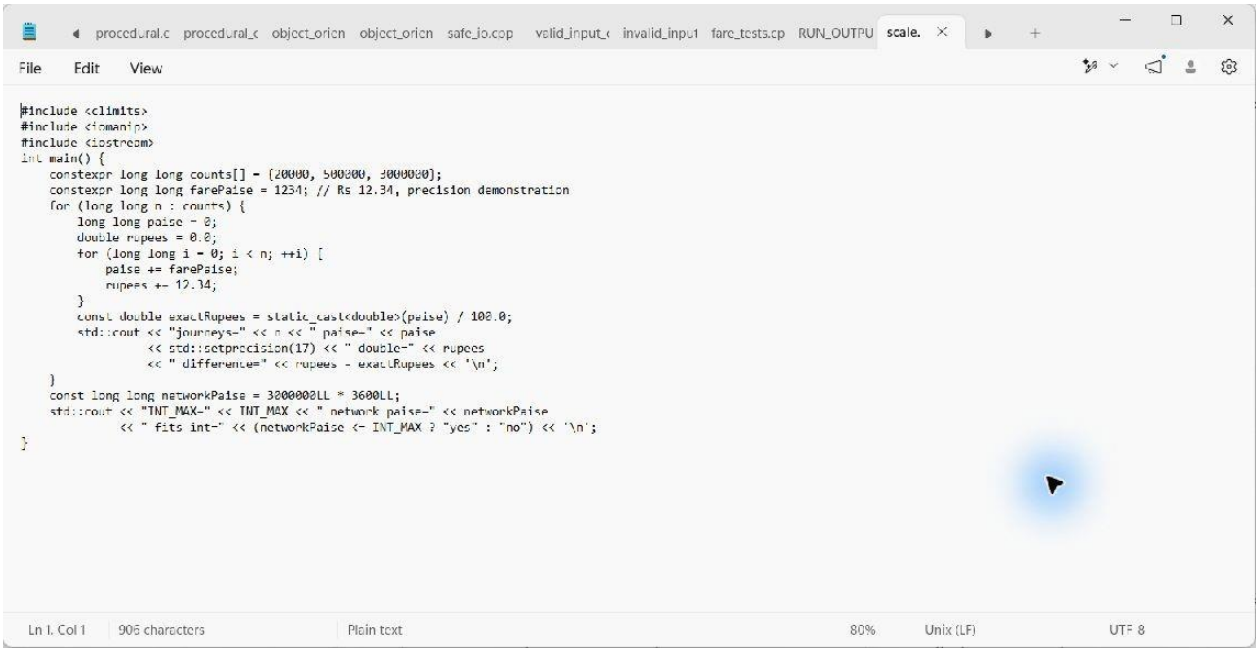
11. Task 10 - Scale and numeric precision

The program adds Rs 12.34 repeatedly using integer paise and `double`. Integer paise stays exact within its range; repeated binary floating-point addition is approximate.

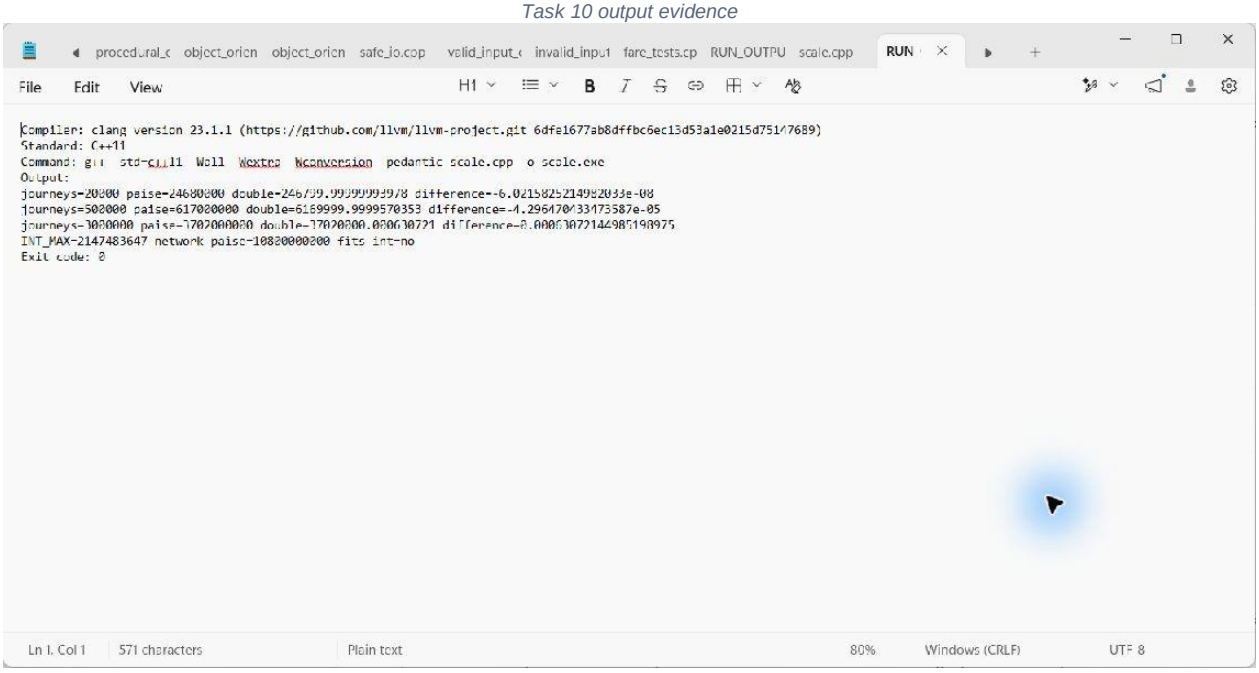
Journeys	Exact total in paise	`double` error in rupees
20,000	24,680,000	-0.0000000602
500,000	617,000,000	-0.0000429647
3,000,000	3,702,000,000	+0.0006307215

The separate maximum-fare scenario is 3,000,000 x Rs 36 = 10,800,000,000 paise. That exceeds `INT_MAX`, so `long long` is appropriate for the accumulated total. Summing `n` trips takes O(n) time.

Task 10 source evidence



Task 10 output evidence



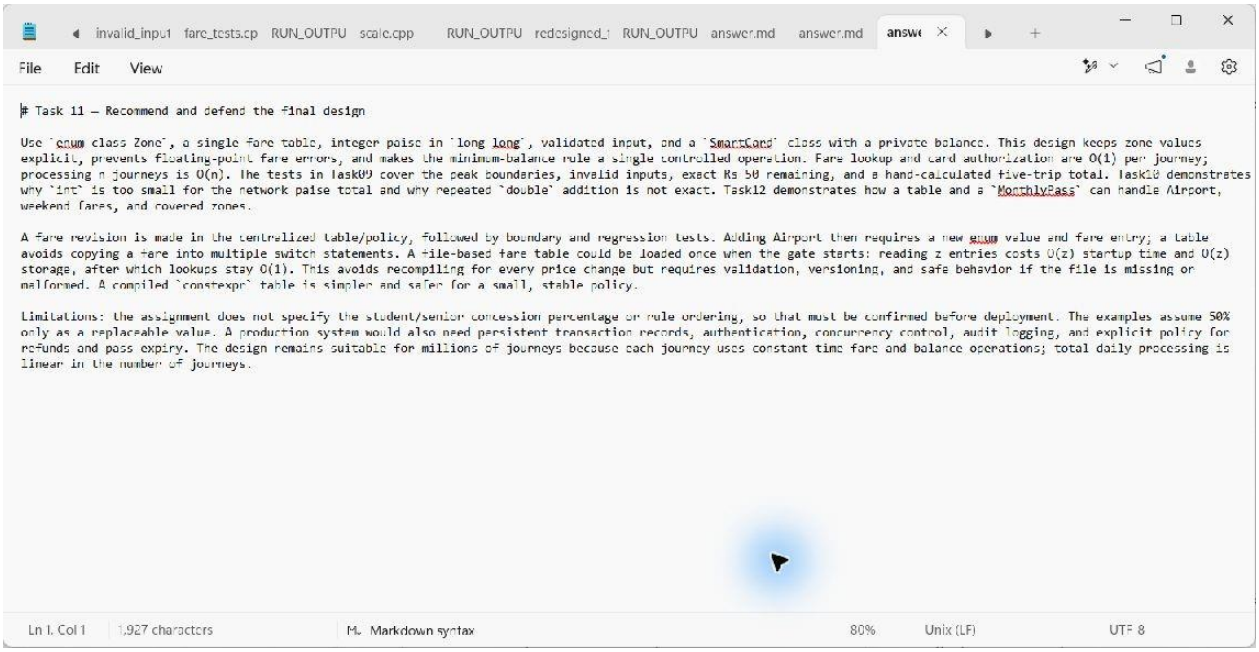
Source: Task10/scale.cpp. Detailed answer: Task10/answer.md.

12. Task 11 - Final design recommendation

Use `enum class Zone`, one fare table, integer paise in `long long`, validated input, and a `SmartCard` with a private balance. This keeps fare lookup and card approval O(1) per trip. The tests in Task 9 check important boundaries and invalid inputs; Task 10 shows why `int` is too small for the largest network total.

A production system would also need persistent transaction records, authentication, concurrency controls, and clear rules for concessions, refunds, and pass expiry. The handout does not specify the concession percentage, so that rule must be confirmed before use.

Task 11 answer evidence



Detailed answer: Task11/answer.md.

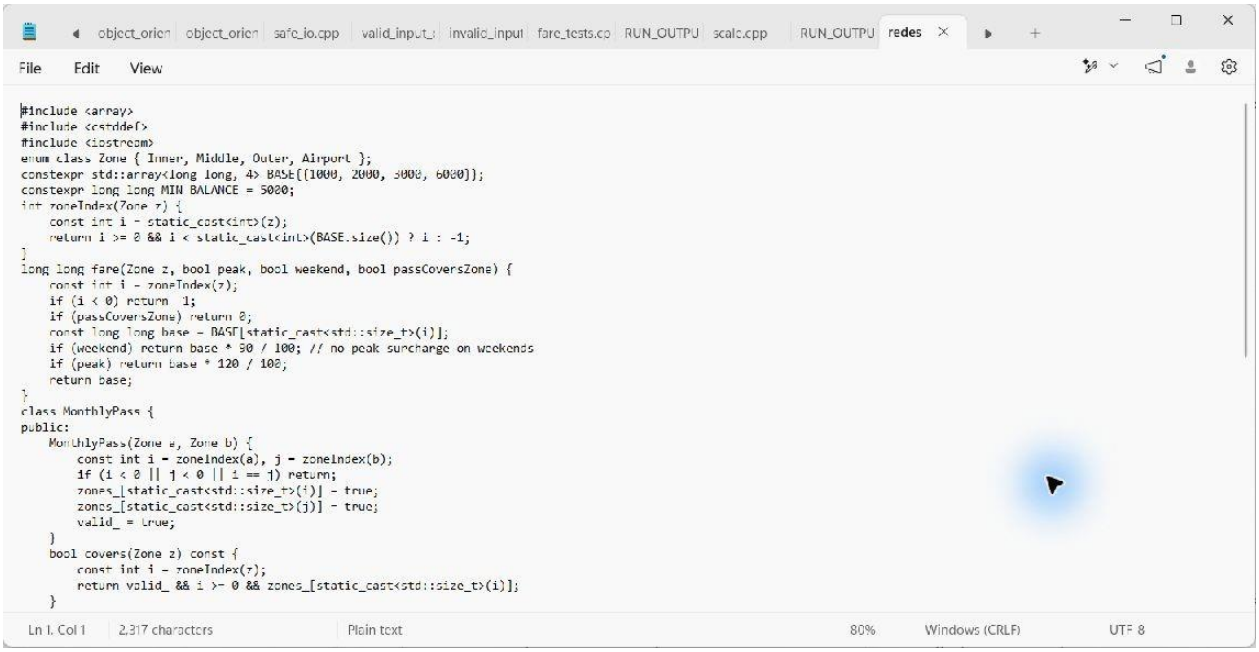
13. Task 12 - Design challenge

Airport is added at Rs 60. The weekend discount is 10% and replaces the peak surcharge. A monthly pass covers two selected zones. The `SmartCard` still protects the minimum-balance rule.

Example	Calculation	Fare
Airport weekday peak	6000 x 120%	7200 paise (Rs 72)
Airport weekend during peak time	6000 x 90%; no peak surcharge	5400 paise (Rs 54)
Airport covered by the pass	Pass covers the zone	0 paise

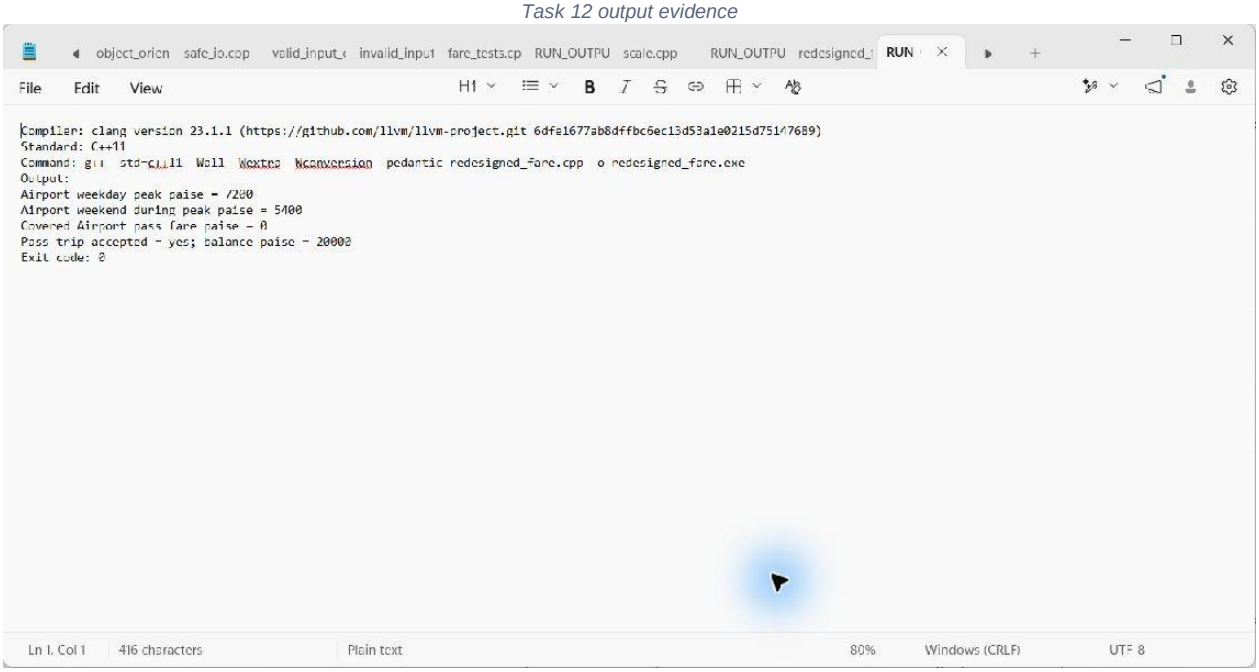
The program accepts the covered pass trip and keeps the balance at 20,000 paise. Fare lookup and checking the two-zone pass are O(1). Pass expiry and renewal still need rules.

Task 12 source evidence



```
#include <array>
#include <cstddef>
#include <iostream>
enum class Zone { Inner, Middle, Outer, Airport };
constexpr std::array<long long, 4> BASE{{1000, 2000, 3000, 6000}};
constexpr long long MIN_BALANCE = 5000;
int zoneIndex(Zone z) {
    const int i = static_cast<int>(z);
    return i >= 0 && i < static_cast<int>(BASE.size()) ? i : -1;
}
long long fare(Zone z, bool peak, bool weekend, bool passCoversZone) {
    const int i = zoneIndex(z);
    if (i < 0) return -1;
    if (passCoversZone) return 0;
    const long long base = BASE[static_cast<std::size_t>(i)];
    if (weekend) return base * 90 / 100; // no peak surcharge on weekends
    if (peak) return base * 120 / 100;
    return base;
}
class MonthlyPass {
public:
    MonthlyPass(Zone a, Zone b) {
        const int i = zoneIndex(a), j = zoneIndex(b);
        if (i < 0 || j < 0 || i == j) return;
        zones_[static_cast<std::size_t>(i)] = true;
        zones_[static_cast<std::size_t>(j)] = true;
        valid_ = true;
    }
    bool covers(Zone z) const {
        const int i = zoneIndex(z);
        return valid_ && i >= 0 && zones_[static_cast<std::size_t>(i)];
    }
```

Task 12 output evidence



```
Compiler: clang version 23.1.1 (https://github.com/llvm/llvm-project.git 6dfe1677ab8dffbc6ec13d53a1e0215d751f7689)
Standard: C++11
Command: g++ -std=c++11 -Wall -Wextra -Wconversion -pedantic redesigned_fare.cpp -o redesigned_fare.exe
Output:
Airport weekday peak paise = 7200
Airport weekend during peak paise = 5400
Covered Airport pass fare paise = 0
Pass trip accepted = yes; balance paise = 20000
Exit code: 0
```

Source: Task12/redesigned_fare.cpp. Detailed answer: Task12/answer.md.

Build, results, and evidence

The C++ programs were built with C++11 and warning flags. The recorded builds and runs succeeded for Tasks 03-10 and 12. Task 7 contains two separate programs. Task 4 produces one documented signed/unsigned comparison warning; Task 9 reports zero failures.

The screenshots above show selected source and result evidence. All task answers, source files, input files, run records, and the remaining screenshots are stored in the assignment folders. Output screenshots show saved run transcripts open in Notepad, not live terminal captures. The Task 4 and Task 5 prediction files are reference predictions prepared with assistance.

Conclusion

The design uses integer paise, named zones, input validation, centralized fare rules, and a private card balance. Recorded runs cover the fare boundaries, invalid input, balance checks, numeric precision, and the Airport/pass design change.

Preparation note: Digital coding and writing assistance was used to prepare parts of this report and the programs. The student should review the work and follow the course rules before submitting it.